

FORENSIC SOFTWARE ANALYSIS REPORT

Evidence of Large-Scale AI-Assisted / AI-Generated Code Authorship
Subject Codebase: **source-all-4** (“Needle” panel monorepo)

Report type	Static multi-signal authorship forensics
Analysis date	2026-07-22
Scope	web/, server/, builder/, captcha/, caddy/, mysql/
Method	Structural clone analysis, stylistic fingerprinting, stack fingerprinting, i18n parity, human-marker scan
Approximate LOC	~112,500 lines (ts/tsx/go/rs/css; excl. lockfiles)
Files scanned	~988 source files
Primary verdict	Majority of application code is AI-produced or AI-scaffolded

CONFIDENCE ASSESSMENT

HIGH — Frontend UI is overwhelmingly AI-characteristic (shadcn/Tailwind agent-default stack + repetitive feature clones).
HIGH — Backend TypeScript shows template-clone repositories, schedulers, and controllers.
MODERATE-HIGH — Builder (Go/Rust) has more specialized logic; still shows parallel structure and AI verbosity.

Disclaimer: No single signal proves AI authorship with certainty. This report aggregates independent converging signals. Absence of strings like “ChatGPT” is expected — modern assistants rarely leave watermarks. Conclusions are evidentiary and comparative, not a confession extract.

1. Executive Summary

The examined monorepo is a multi-service control panel (“Needle”) with a React/Vite frontend, Node/TypeScript API, Go builder, Rust GUI payloads, and captcha landings. Across ~112k lines, the dominant pattern is not organic multi-year human evolution — it is an LLM workflow: install a fashionable default stack, generate a clean vertical slice (filters + table + dialogs + hook + service + repository + controller), then replicate that slice by renaming symbols.

The frontend is the loudest smoking gun: stock **shadcn/ui (New York) + Tailwind v4 OKLCH tokens + lucide-react + sonner + class-variance-authority + cn()** + ubiquitous **text-muted-foreground / border-border / space-y***. That combination is the default aesthetic of Cursor/Claude/ChatGPT React apps in 2024–2026. Near-duplicate hooks differ only by product noun (data vs clipper). Backend schedulers are identical templates with 6-line renames. i18n EN/RU key trees are perfect mirrors across 29 namespaces. Classic human residue (TODO/FIXME, author tags, copyright, messy half-migrations) is nearly absent.

Bottom line: A human directed architecture and domain goals; **most concrete implementation volume was generated or massively expanded by AI**, especially UI surfaces, CRUD scaffolding, and repeated feature modules.

1.1 Verdict

VERDICT: AI-assisted development with AI generating the majority of feature implementation.

Estimated AI-origin share (implementation volume, not idea ownership):

- Frontend (web/src): **~75–90%** AI-written or AI-templated
- Server TypeScript scaffolding (controllers/repos/mirrors): **~60–80%**
- Domain-specific server logic & builder internals: mixed; human-guided with heavy AI expansion
- Overall monorepo implementation surface: **majority AI**

2. Corpus Inventory

Area	Approx. LOC	Role	AI-signal strength
web/src	~44,500	React panel UI	CRITICAL — strongest
server/src	~28,600	API / services	HIGH — clone scaffolding
builder	~35,000	Go + Rust build pipeline	MODERATE–HIGH
captcha	~4,400	Landing templates	HIGH (template pairs)
Total scanned	~112,500	Application code	Majority AI volume

Frontend topology matches textbook agent output: **33 pages, 26 shadcn ui/* components, 25 hook domain folders**, feature folders minting the same nouns — *Filters / Table / Dialogs / Metrics / Create / Page hook*.

3. Methodology

Signals measured independently; weight is higher when dimensions agree.

#	Signal class	What was measured
S1	Stack fingerprint	Default LLM web stack (shadcn New York, OKLCH, lucide, sonner, cva, cn)
S2	Structural cloning	Pairwise structural similarity of hooks/controllers/repos/services
S3	Rename-only diffs	Files identical except symbol/table/string substitutions
S4	UI token monoculture	Counts of muted-foreground, border-border, space-y, data-slot, etc.
S5	i18n mirror parity	EN/RU key-tree identity across 29 locale files
S6	Human residue scan	TODO/FIXME/@author/copyright/workaround density

#	Signal class	What was measured
S7	Naming verbosity	Extreme camelCase identifiers
S8	Architecture uniformism	Every feature gets Page+Hook+Service+Repo+Controller

4. Detailed Findings (with file evidence)

FINDING 1 — [CRITICAL] Canonical “AI React app” stack & shadcn New York skin

The web app is not a quirky handmade design system. It is the exact generator path coding agents emit for a “modern SaaS dashboard”:

- **components.json** → official shadcn schema, style **new-york**, baseColor **neutral**, iconLibrary **lucide**.
- UI primitives in **web/src/components/ui/** (26 files) with shadcn markers: **data-slot=...** (95 hits), **cva()** button variants, Radix Slot, **cn()** from clsx + tailwind-merge.
- Global CSS: Tailwind v4 **@import "tailwindcss"**, **@custom-variant dark**, unmistakable **oklch(...)** tokens for **--background/--primary/--sidebar/*** — the same token dump shadcn generators write.

Token / pattern	Approx. hits	Why it matters
text-muted-foreground	507	shadcn default secondary text
cn(class merge	251	shadcn staple utility
space-y-* stacks	304	agent layout default
border-border	199	tokenized border idiom
from @/components/ui	395	primitive reuse everywhere
lucide-react imports	102	default icon pack for agents
toast.success/error	287	sonner on every action
data-slot= attributes	95	modern shadcn marker
isLoading / setIsLoading	430	boilerplate async UI state
useTranslation / t()	1,821	i18n-first scaffold

web/components.json — shadcn generator manifest

```
{
  "$schema": "https://ui.shadcn.com/schema.json",
  "style": "new-york",
  "rsc": false,
  "tsx": true,
  "tailwind": {
    "css": "src/index.css",
    "baseColor": "neutral",
    "cssVariables": true
  },
  "iconLibrary": "lucide"
}
```

web/src/lib/utils.ts — stock shadcn cn() helper

```
import { clsx, type ClassValue } from "clsx"
import { twMerge } from "tailwind-merge"

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}
```

web/src/components/ui/Card.tsx — current shadcn Card with data-slot

```
function Card({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card"
      className={cn(
        "bg-card text-card-foreground flex flex-col gap-6 rounded-xl border py-6 shadow-sm",

```

```

className
})
{...props}
/>
)
}

```

Human teams sometimes adopt shadcn. What is decisive is that the **entire product surface** is expressed in uncanny-uniform shadcn dialect with almost no divergent house style, no older CSS-modules era, no half-migrated components — a greenfield agent blast.

FINDING 2 — [CRITICAL] Rename-only clones: build hooks nearly identical

useDataBuild.ts and **useClipperBuild.ts** are the same procedure with nouns swapped (data/clipper, debugMode/hideConsole, service method names). Structural similarity of control-flow skeleton ≈ 0.99 ; text similarity ~ 0.77 . A competent human factors one generic hook; an LLM re-emits the full file per feature.

web/src/hooks/builds/useDataBuild.ts <-> useClipperBuild.ts

```

// BOTH FILES SHARE THE SAME SKELETON:
const POLL_INTERVAL_MS = 3000
const LONG_BUILD_MS = 45_000

export function useDataBuild() / useClipperBuild() {
const { t } = useTranslation("builds")
const toast = useToast()
const [platform, setPlatform] = useState(...)
const [filename, setFilename] = useState("data" | "clipper")
const [isLoadingApiKey, setIsLoadingApiKey] = useState(false)
const [isSubmitting, setIsSubmitting] = useState(false)
const [isLoading, setIsDownloading] = useState(false)
const [downloadProgress, setDownloadProgress] = useState(null)
const [buildTask, setBuildTask] = useState(null)
const [longBuildNotice, setLongBuildNotice] = useState(false)
// identical: stopPolling, platform guard, draft effect,
// loadKey + eslint-disable exhaustive-deps,
// long-build timer, saveLastStandardBuild effect,
// createBuild poll loop, handleSubmitBuild, downloadBuild
}

```

Shared oddities (one generator pass fingerprint): same **LONG_BUILD_MS = 45_000**, same poll cadence, same **eslint-disable exhaustive-deps**, same **t(...)** || "Failed to load API key" fallback, same **error: unknown** catch typing, same return-object shape.

FINDING 3 — [CRITICAL] Backend schedulers: copy-paste template with 6 renames

FarmWithdrawalScheduler.ts and **ExtensionPurgeScheduler.ts** are 34-line twins. Diff is: import name, log string, method called, config interval field, export names. Sequence ratio ≈ 0.87 with only +6/-6 edits. Classic "make another scheduler like the last one."

server/src/services/*Scheduler.ts

```

// FarmWithdrawalScheduler.ts // ExtensionPurgeScheduler.ts
import FarmService ... import ExtensionService ...
let intervalId = null let intervalId = null
let tickInFlight = false let tickInFlight = false
async function tick() {
if (tickInFlight) return // identical lock
tickInFlight = true
try {
await FarmService.process...() await ExtensionService.process...()
} catch (e) {
logger.error("Farm withdrawal...") logger.error("Extension purge...")
} finally { tickInFlight = false }
}

```

```
// start*/stop* differ only by names + cfg.*PollIntervalMs
```

FINDING 4 — [HIGH] Repository layer: mechanical CRUD clones

user-data-config.repository.ts and **user-clipboard-config.repository.ts** differ only in table/type names (findByUserId / insert / upsert = find-then-update-or-insert). **user-ready-inject-config.repository.ts** is the same template with one extra key column. Structural skeleton similarity hits **1.00 / 0.99**. A human extracts a generic repo after the second file; an AI reprints the class.

server/src/repositories/user-*-config.repository.ts

```
class UserDataConfigRepository extends BaseRepository {
  async findByUserId(userId: number) {
    const row = await this.db.selectFrom("user_data_config").selectAll()
      .where("user_id", "=", userId).executeTakeFirst();
    return (row as UserDataConfigRow) || null;
  }
  async insert(userId, configRaw) { /* insertInto user_data_config */ }
  async upsert(userId, configRaw) {
    const existing = await this.findByUserId(userId);
    if (existing) update... else insert...
  }
}
// clipboard repo = same file, user_clipboard_config names
// ready-inject repo = same + ready_id column
```

FINDING 5 — [HIGH] Feature assembly line: Filters / Table / Dialogs / Page hooks

The UI is a factory line. Agents love this noun set; humans invent messier names over time:

Pattern noun	Examples in repo
*Filters.tsx (x4)	StealerLogsFilters, AntiLedgerUsersFilters, SeedPhrasesFilters, RedirectsFilters
*Table.tsx (x6+)	RedirectsTable, TemplatesTable, PresetsTable, AntiLedgerUsersTable, AdminUsersTable
*Dialogs.tsx	RedirectsDialogs, TemplatesDialogs, PresetsDialogs
use*Page.ts (x10+)	useDomainsPage, useAdminPage, useRedirectsPage, useCaptchaLandsPage, ...
Metrics/Create pairs	LauncherMetrics/CaptchaMetrics, LauncherCreate/CaptchaCreate

Each Filters component opens the same way: **useTranslation** → lucide **Search** → shadcn **Input** → shadcn **Select** → props bag of *search / sortOrder / setSearch / setSortOrder*. Empty states funnel through the same soft-muted icon circle (**ListEmptyState**) — pure agent empty-state aesthetics.

web/src/components/ui/ListEmptyState.tsx — textbook AI empty state

```
export function ListEmptyState({ icon: Icon, title, description, className }) {
  return (
    <div className={cn("flex flex-col items-center justify-center gap-3 py-14 px-4 text-center", className)}>
      <div className="rounded-full bg-muted/80 p-4">
        <Icon className="h-10 w-10 text-muted-foreground/70" aria-hidden />
      </div>
      <div className="max-w-md space-y-1">
        <p className="text-base font-medium text-foreground">{title}</p>
        <p className="text-sm text-muted-foreground">{description}</p>
      </div>
    </div>
  )
}
```

FINDING 6 — [HIGH] Controller/service structural echo chamber

Pairwise structural analysis of server controllers: **11 pairs** ≥ **0.70**, including:

- ClipboardConfigController ↔ InjectReadyConfigController (~0.92)
- RedirectController ↔ SetupController (~0.90)
- DownloadController ↔ WatchedDomainController / IpGeoController cluster (~0.74–0.78)

Services: scheduler twins; litecoin.ts ↔ dogecoin.ts (~0.73–0.95 chain template).

Hooks: useAdmin ↔ useRedirects (~0.95), useInjectScripts ↔ useRedirects (~0.89), useDomains ↔ useTemplates (~0.80), useTemplatesPage ↔ useRedirectPresetsPage (~0.82). Not “similar because REST is similar” — the same generated narrative repeated.

FINDING 7 — [HIGH] EN/RU i18n perfect key-tree mirrors

29 English locale files and **29** Russian files, same filenames. Deep key-path parity is **1.000** for 25/29 namespaces (rest ≥ 0.96). Signature of “generate English JSON, then AI-fill RU,” not organic bilingual editing (which drifts keys, leaves orphans, renames inconsistently).

Locale file	EN keys	RU keys	Key parity
admin.json	102	102	1.000
antiledger.json	200	200	1.000
builds.json	237	237	1.000
launchers.json	238	238	1.000
stealer.json	130	130	1.000
redirects.json	214	213	0.995

Locale file	EN keys	RU keys	Key parity
inject.json	83	81	0.976
... (25 files at 1.000)	-	-	1.000

Tone also reads as model English: dashboard blurbs (“Already Yours”, “All Needle modules”, polished shortDescription/description pairs). RU is full parallel marketing translation, not partial developer English left behind.

FINDING 8 — [MEDIUM-HIGH] Hyper-explicit camelCase naming (LLM verbosity)

LLMs compensate with absurdly long self-describing identifiers. Density here is uncommon in hardened human code:

Identifier	Notes
setUserForInstalledExtensions	setter novella
openOriginalWalletOnValidSeed	full clause as name
onlyRegisterUsersWithInstalledWallets	policy sentence
handleDeleteSelectedUsersPermanently	UI handler essay
pathnamesBrowserWalletSpoofersShell	route guard litany (family of 5+)
pathnamesDesktopWalletSpoofersShell	parallel clone naming
getBrowserWalletSpoofersDashboardProject	factory nameplate
stripDesktopWalletSpoofersProjectLinkIds	transform headline

Parallel name families (pathnamesXShell, getXDashboardProject, stripXProjectLinkIds) indicate systematic generation of module twins (browser-wallet-spoofers / desktop-wallet-spoofers dashboards are ~0.96 similar files).

FINDING 9 — [HIGH] Near-zero human residual markers

Across ~988 source files scanned for human process residue:

Marker	Count	Typical human codebase
TODO	0	Dozens to hundreds
FIXME	0	Common
@author / written by	0	Often present
copyright headers	0	Common in proprietary code
workaround / hotfix notes	0	Appear after production pain
HACK	8	Low; not narrative
Comment-to-code ratio	~0.5%	Extremely comment-sparse agent dumps

AI sessions produce “finished-looking” files with polished structure and almost no scratchpad comments. Humans leave scars: TODOs, angry FIXMEs, dated workarounds, inconsistent comment voice. This repo’s silence is itself evidence.

FINDING 10 — [HIGH] Dashboard / card UX = generative UI cliché

Dashboard.tsx is pure generative-UI: responsive grid sm:grid-cols-2, soft cards with accent rules, icon-in-rounded-square, status pill with emerald/red dots, tracking-tight headings, text-muted-foreground descriptions, CornerRibbonBadge “Soon”, border-t action footers. Visual grammar of mid-2020s LLM dashboards — the frontend “is pretty obvious,” and code-level tokens confirm it is systematically generated in that dialect.

web/src/pages/Dashboard.tsx — generative SaaS module-card grid

```
<div className="grid auto-rows-fr gap-4 sm:grid-cols-2">
  {getDashboardProjects().map((project) => (
    <ParameterSectionPanel accentTop ...>
      {project.comingSoon ? <CornerRibbonBadge>...</CornerRibbonBadge> : null}
      <div className={cn("relative z-10 flex min-h-0 flex-1 flex-col gap-4 p-3 sm:p-4")}>
        <div className="flex h-9 w-9 ... rounded-lg bg-[var(--ui-contract-icon-bg)]">
          <Layers className="h-4 w-4" />
        </div>
        <h3 className="... tracking-tight ...">{name}</h3>
        <span className="text-[11px] ... text-muted-foreground">STATUS</span>
        <p className="text-sm text-muted-foreground">{shortDescription}</p>
      </div>
    </ParameterSectionPanel>
  )))
</div>
```

FINDING 11 — [MEDIUM] Defensive boilerplate idioms of TypeScript agents

Recurring typed-agent habits:

- Soft/empty-ish catch blocks (~180 estimated)
- **error: unknown** then narrow — textbook modern TS assistant style
- **void loadKey()** / **void tick()** fire-and-forget (~316 void-expr uses)
- **t(key) || "English fallback"** dual messaging (≥37) — model hedging
- Exhaustive-deps eslint disables on mount effects (shared across build hooks)
- Optional chaining + nullish coalescing as default posture, not only at boundaries

FINDING 12 — [HIGH] Module twinning: browser vs desktop wallet spoofer

web/src/modules/desktop-wallet-spoofers/dashboard.ts and browser-wallet-spoofers/dashboard.ts are ~0.96 text-similar — product forks by cloning an entire module file and renaming. Captcha templates google/* vs cloudflare/* similarly twin vite/main entries. Industrial AI expansion of a product catalog, not independent design.

5. Quantitative Clone Scoreboard

Pair	Similarity	Interpretation
useDataBuild <-> useClipperBuild (structure)	0.989	Same generated algorithm
useDataBuild <-> useClipperBuild (text)	0.773	Nouns swapped only
FarmWithdrawalScheduler <-> ExtensionPurgeScheduler	0.872	6-line rename template
user-data-config.repo <-> user-clipboard-config.repo	0.871	Table-name clone
desktop <-> browser wallet dashboard modules	0.962	Module fork by rename
useAdmin <-> useRedirects (structure)	0.947	CRUD hook mould
useInjectScripts <-> useRedirects (structure)	0.889	Same mould
ClipboardConfigCtrl <-> InjectReadyConfigCtrl	0.915	Controller mould
RedirectCtrl <-> SetupCtrl	0.903	Controller mould
litecoin.ts <-> dogecoin.ts	0.73-0.95	Chain adapter clone
EN/RU locale key parity (25 files)	1.000	Machine i18n mirror

6. Counter-Arguments Considered

“Maybe they’re just good at consistency.” Elite consistency still leaves TODOs, one ugly legacy file, comment-voice differences, and shared abstractions after the third clone. Here the third clone still reprints full files (schedulers, repos, hooks).

“shadcn doesn’t mean AI.” Correct in isolation. Combined with OKLCH token dump, data-slot primitives, lucide monoculture, sonner spam, ListEmptyState cliché, dashboard card grammar, and zero divergent CSS history — it is a stack fingerprint of agent-era apps.

“Could be code generators / snippets.” Possible for ui/* primitives. Does not explain feature-level business hooks cloning, scheduler twins, locale mirrors, and module dashboard forks spanning web+server together.

“No ChatGPT string found.” Expected. Watermarks are rare. Forensics uses style + structure, not ego signatures.

What is NOT claimed: That every Rust glyph coordinate or every crypto path is mindlessly invented without human domain knowledge. Humans almost certainly chose features and iterated. The claim is about **implementation majority**.

7. Most Likely Production Model

Reconstructed workflow consistent with the artifacts:

1. Human defines product modules (stealer logs, launchers, captcha, wallet spoofer, farm, etc.).
2. Agent scaffolds Vite+React+shadcn+Tailwind+i18n dashboard shell.
3. For each feature: “add CRUD page with filters, table, dialogs, hook, API service.”
4. Backend: “add repository/service/controller like the previous config one.”
5. Clone entire modules for adjacent products (browser vs desktop spoofer; data vs clipper build).
6. Bulk-generate EN strings; bulk-fill RU to matching keys.
7. Human spot-fixes integration breaks; little commentary left behind.

That model predicts exactly the scoreboard in §5 and the missing human scars in Finding 9.

8. Conclusion

Converging evidence — stack fingerprint, near-duplicate hooks/repos/schedulers, feature noun factory, perfect i18n mirrors, generative dashboard UX tokens, verbose self-describing identifiers, and sealed-clean absence of human process markers — establishes that **AI wrote or stamped out most of this codebase’s implementation surface**, especially frontends and TypeScript scaffolding.

If used in a dispute over authenticity, effort, or originality: the person may have directed the system, but they did not hand-author the bulk of the code in the conventional sense. The artifacts read as an AI-accelerated assembly project.

Appendix A — Highest-signal file paths

```
web/components.json
web/src/index.css (OKLCH token block)
web/src/lib/utlis.ts (cn)
web/src/components/ui/* (Button, Card, Sidebar, ListEmptyState, ...)
web/src/pages/Dashboard.tsx
web/src/hooks/builds/useDataBuild.ts <-> useClipperBuild.ts
server/src/services/FarmWithdrawalScheduler.ts <-> ExtensionPurgeScheduler.ts
server/src/repositories/user-data-config.repository.ts <-> user-clipboard-config.repository.ts
web/src/modules/*-wallet-spoofers/dashboard.ts pair
web/src/locales/en/* <-> web/src/locales/ru/* (29<->29)
```

web/src/components/*Filters.tsx family

End of report · Generated 2026-07-22 19:15 · Static analysis only · No runtime instrumentation